

Complaints Backend Breakdown

The Complaints module is built and routed on the frontend — 42 components across 26 routes — and every one of them runs on mock data. This splits the work into eleven backend modules that can be picked up and finished one at a time, in an order where nothing waits on something that hasn't been built yet.

WHERE THINGS STAND

No complaints endpoint exists yet. Every screen reads from a local file under `src/Components/ComplaintsComps/`, and each of those files states the exact JSON shape its screen expects — they were written to be replaced by a response, field for field. The shapes quoted below come from those files, so they are the contract, not a proposal.

Build order

Five phases. Each phase only depends on the ones before it, so a phase can ship, be tested against the real screens, and be signed off before the next begins.

PHASE 1

Foundation

M1 · M2

PHASE 2

Records & lifecycle

M3 · M4 · M5

PHASE 3

Routing & timing

M6 · M7

PHASE 4

Surfaces

M8 · M9

PHASE 5

Cross-cutting

M10 · M11

Phase 1 — Foundation

Nothing else can be built honestly until these two exist: every other module references a category, and every screen is gated on a permission.

M1

Categories

The category master for both streams — parent complaints and staff concerns keep separate lists.

SCREENS IT POWERS

- Complaint Categories (parent)
- Internal Categories (staff concerns)
- Create Category dialog · Add Category drawer

ENDPOINTS

[list by stream](#)[create](#)[update](#)[set status](#)[delete](#)

- Both intake forms only ever offer **status:** **ACTIVE** categories — the filter is already in the frontend.

RECORD

```
id · name · description · priority CRITICAL | HIGH | NORMAL | LOW · owner (role name) ·
status ACTIVE | INACTIVE
internal stream adds: department · sla
```

DEPENDS ON **Nothing — start here**

M2

Roles & permissions

Complaints RBAC, delivered through the existing permission envelope rather than a new mechanism.

SCREENS IT POWERS

- Complaint Permissions · Internal Permissions
- Every complaints route — all 26 are wrapped in an access gate

ENDPOINTS

roles + user counts users by role

permission map save map

- Return `mainMenu: "complaints"` from the existing permissions call, with the subMenu keys listed at the end of this page.

PERMISSION KEYS THE SCREENS TOGGLE

Parent (11) — view · register · assign · reassign · updateStatus · internalNotes · requestInfo · resolve · escalate · confidential · close

Internal (19) — viewOperations · createEntry · assignEntry · reassignEntry · updateStatus · internalNotes · addReviewer · approveEntry · resolveEntry · reopenEntry · escalateEntry · viewReports · viewDashboard · manageCategories · manageAssignment · manageSla · manageEscalation · manageNotifications · viewAuditLog

DEPENDS ON

Nothing. Note the gate currently allows everything — this module is what switches enforcement on, with no frontend change needed.

Phase 2 — Records & lifecycle

The two ways a record is created, and everything that happens to it afterwards. M5 is the largest single piece of work in the module.

M3

Parent complaint intake

Office staff register a complaint on a parent's behalf, against a student on record.

SCREENS IT POWERS

- Register Complaint — student search
- Register Complaint — intake form

ENDPOINTS

`student search` `create complaint`

- Search by name or admission no, filtered by class and section, paginated.
- Create is multipart — attachments ride along with the form.
- Response must return the generated reference; the screen navigates to it.

STUDENT SEARCH RESULT

```
id · name · admissionNo · grade ·  
section · parentName · parentMobile
```

CREATE PAYLOAD

```
studentId · source "Phone Call" |  
"Walk-In" · category · subject ·  
statement · internalNotes ·  
immediateAction · confidential bool ·  
receivingStaff · attachments[]
```

DEPENDS ON M1 (category list) · M2 (`register` permission)

M4

Staff concern intake

A staff member raises an internal operations issue. No student is attached.

SCREENS IT POWERS

- Add Issue
- Add New picker (chooses between this and M3)

ENDPOINTS

`create issue`

- Returns an `ACT` reference — see the reference formats below.
- Attachments up to 10 MB here, against 5 MB on the parent form.

CREATE PAYLOAD

```
category · title · description · location · attachments[] · raised-by resolved from the  
actor
```

DEPENDS ON M1 (internal category list) · M2 (`createEntry`)

M5

Workspace & lifecycle

The unified queue, the detail record, and every action that moves a complaint forward. The core of the module.

SCREENS IT POWERS

- Complaints Management workspace
- Complaint Detail · Action Detail
- Update Status drawer · Timeline

ENDPOINTS

`list (both streams)` `detail by ref`

`update status` `add note`

`assign / reassign` `escalate`

`request info` `resolve` `close`

LIST ROW

```
id · ref · type "Parent Complaint" |
"Internal Excellence" · title · student
(parent only) · owner · priority ·
status · sla · slaTone
overdue|due|ok|neutral · date
```

- Tabs: All · Parent Complaints · Internal Excellence, each with its own pagination.

DETAIL RECORD

```
ref · stream · category · priority ·
status · title · registeredAt · slaDue
· student[] · parent[] · description ·
attachments[] · internalNotes[] {text,
author, at} · resolution (null until
resolved) · assignment[] · timeline[]
{label, at, state}
```

STATE VOCABULARY — THE SCREENS COLOUR-CODE THESE EXACT STRINGS

status — Registered · Action Required · Under Review · In Progress · Awaiting Review · Resolved · Closed

status update options — Action in Progress · Resolved · Escalated · On Hold

timeline state — done | active | pending · parent runs 8 steps, Registered » Closed

update payload — {status, note, notify, expectedResolutionDate}

DEPENDS ON M3 · M4 (records to list) · M2 (per-action permissions)

Phase 3 — Routing & timing

Who a new record goes to, and when it is late. Both are currently hardcoded strings on the frontend —

"Due Today", "Overdue" — with no logic behind them.

M6

Assignment mapping

Category » role » owner, per stream, with auto and manual modes.

SCREENS IT POWERS

- Complaint Assignment Mapping
- Internal Assignment Mapping
- Mapping drawer (add / edit a rule)

ENDPOINTS

list mappings

create

update

set status

- On create in M3/M4, resolve the owner from this table when the mode is Auto Assign.

RECORD

id · category · role · owner · mode "Auto Assign" | "Manual Assignment" · status Active | Inactive

DEPENDS ON M1 (categories) · M2 (roles)

M7

SLA & escalation

Deadlines per priority band, and the ladder a record climbs when it breaches them.

SCREENS IT POWERS

- SLA Configuration (parent + internal)
- Escalation Configuration
- Every SLA cell and countdown in M5 and M8

ENDPOINTS

get SLA rules

save SLA rules

get escalation

save escalation

- Plus the engine itself: compute due dates on create, and drive `slaTone` on every list row.

SLA RULES

four priority bands × five deadlines –
acknowledgement · **assignment** ·
initialResponse · **resolution** · **closure**
 plus working-hours rules and pause
 toggles

ESCALATION

levels[] {level, role, description} –
 school-configurable, roles come from
 user types
triggers[] {key, title, description,
 enabled} – SLA breached, critical
 action, and the rest

DEPENDS ON M1 (priority per category) · M5 (records to time)

Phase 4 — Surfaces

Two read-heavy views over data the earlier phases already produce. Neither introduces a new record type; both need aggregation.

M8

My Work

The staff-facing queue — only what is assigned to the signed-in user.

SCREENS IT POWERS - My Work queue (two tabs, with counts) - My Work item detail	ENDPOINTS my queue by stream item detail mark completed - Scoped by assignee, filtered by priority, status and free text. - Tab counts come from the response, not from array length.
--	--

QUEUE ROW

id · ref · priority · category · title · student · grade · status · due · dueUrgent bool

DEPENDS ON

M5 (the records) · M7 (due dates) · M2 (mywork)

M9

Dashboards

Management metrics for both streams, plus the config that decides which cards appear.

SCREENS IT POWERS

- Management Dashboard — parent tab
- Management Dashboard — staff concerns tab
- Dashboard Configuration (both streams)

ENDPOINTS

`parent metrics` `internal metrics`
`widget config` `save widget config`

PARENT METRICS

volume — today, week, month, open, closed
attention — overdue, critical, high priority, unassigned, reopened
breakdowns — by category, class, role, employee, source
plus SLA metrics, frequent complaints, frequently involved, parent satisfaction

INTERNAL METRICS

headline stats · by category · by priority · resolution time · on-time completion · reopened actions · staff actions · compliance overview · repeated issues · activity feed · SLA metrics

WIDGET CONFIG

`widgets[] {key, title, description, enabled}` — saved per stream; **enabled** is what the save call returns

DEPENDS ON M5 · M7 — everything here is an aggregate over those

Phase 5 — Cross-cutting

Both of these hang off events in M1–M9. Design them alongside those modules; ship them once the events they listen to actually fire.

M10

Notification templates

The messages sent on lifecycle events, editable per stream.

SCREENS IT POWERS

- Notification Templates (parent + internal)
- Edit Template dialog

ENDPOINTS

`list templates``update template``set status`

- Parent and internal templates save to separate paths — the screen already sends them separately.
- Delivery itself is triggered by M5; the status drawer sends `notify: true`.

RECORD

```
id · name · channels[] PUSH | SMS | EMAIL · status ACTIVE | INACTIVE · subject · body  
placeholders in use: {{complaint_id}} · {{parent_name}} · {{complaint_category}} ·  
{{sla_days}}
```

DEPENDS ON M5 (events) · M7 (SLA reminders)

M11

Audit log

Who changed what, across every screen in the module.

SCREENS IT POWERS

- Audit Log (parent + internal)
- Audit entry detail

ENDPOINTS

`list, filtered``entry detail`

- Filters on user, action and module; paginated.
- Write side is not an endpoint — every mutation in M1–M10 records an entry.

RECORD

```
id · dateTime · user · role · action · module · reference
```

DEPENDS ON Every module that writes — specify with them, ship last

Conventions that apply to all eleven

Settle these once, at the start. They are assumptions the frontend already makes.

CONCERN	WHAT THE FRONTEND EXPECTS
Reference format	Parent <code>MSMS-CMP-2026-00124</code> , internal <code>MSMS-ACT-2026-0042</code> . Both streams share one detail URL and the screen picks which view to render by matching <code>-ACT-</code> in the reference — so the format is load-bearing, not cosmetic.
Acting user	Send the actor the same way the Leave & Payroll module does — <code>requestedByRollNumber</code> on every request. Worth matching so the two modules behave alike.
Academic year	Every list and metric is scoped to the selected year, as elsewhere in the app.
Pagination	<code>{ pageSize, totalItems, pageCount }</code> alongside the rows. Counts and badges must come from the response, never from array length — the screens are written that way already.
Attachments	Parent intake: 5 MB, <code>.png .jpg .jpeg .pdf .doc .docx</code> . Staff concerns: 10 MB, <code>.pdf .png .jpg .jpeg</code> . Client-side limits are hints; the API is the real gate.
Empty vs error	Return an empty list for "nothing yet", not an error object. A screen that cannot tell the two apart shows a failure toast where it should show an empty state.

Permission subMenu keys

Under `mainMenu: "complaints"` . These twenty are already named in the frontend and wired to routes — returning them is all that is needed to turn enforcement on.

GROUP	KEYS
Module level	dashboard · manage · mywork · configurations
Parent config	categories · rolepermissions · sla · escalation · notificationtemplates · assignmentmapping · dashboardconfig · auditlog
Internal config	internalcategories · internalrolepermissions · internalsla · internalescalation · internalnotificationtemplates · internalassignmentmapping · internaldashboardconfig · internalauditlog

Derived from the mock-data files in `src/Components/ComplaintsComps/` — each one documents the shape its screen expects.

42 components · 26 routes · 2 streams · 0 endpoints built